
task-app 30日間カリキュラム

Day 30

完成版を公開

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
なぜこれを作るのか	3
実装ステップ一覧	6
Step 1: 本番用の環境変数を準備(読む目安: 5分)	7
Step 2: 本番前のローカル最終確認(読む目安: 5分)	11
Step 2.5: セキュリティヘッダーを設定する(読む目安: 6分)	14
Step 3: Git の公開準備(読む目安: 3分)	17
Step 4: Vercel を設定してプッシュする(読む目安: 25分)	19
Step 5: 本番環境の動作確認(読む目安: 7分)	40
Step 6: 30日間の学習サマリー(読む目安: 7分)	44
Step 7: 技術スタックの振り返り(読む目安: 5分)	46
Step 8: 次のステップとリソース(読む目安: 5分)	48
完成コード全体	58
今日のまとめ	61
つまずきポイント	61
今日学んだ用語	62
理解チェック	62
追加課題: 公開先の別ページでもヘッダーを確かめる	63
卒業おめでとうございます	64
次に読むもの	67
奥付	67

前回の振り返り

Day 29 ではユーザー詳細・編集ページを実装しました。管理者がユーザー情報を閲覧・編集できる画面を作り、権限チェックやフォームバリデーションも組み込みました。

今日はいよいよ最終日。完成したアプリをインターネットに公開して30日間の集大成を形にします。

今日のゴール

完成したタスク管理アプリを Vercel へデプロイし、インターネットに公開します。30日間の学習を振り返り、次のステップを考えます。

なぜこれを作るのか

自分のパソコンでしか動かないアプリはまだ「作品」ではありません。公開して初めて世界中の人に使ってもらえるプロダクトになります。

例え話: デプロイは「料理をお店に並べる」ことです。30日間かけて腕を磨き、レシピを覚え、食材を選びました。

ようやく完成した一皿をテーブルに出す瞬間が一番の醍醐味です。

30日間の歩み

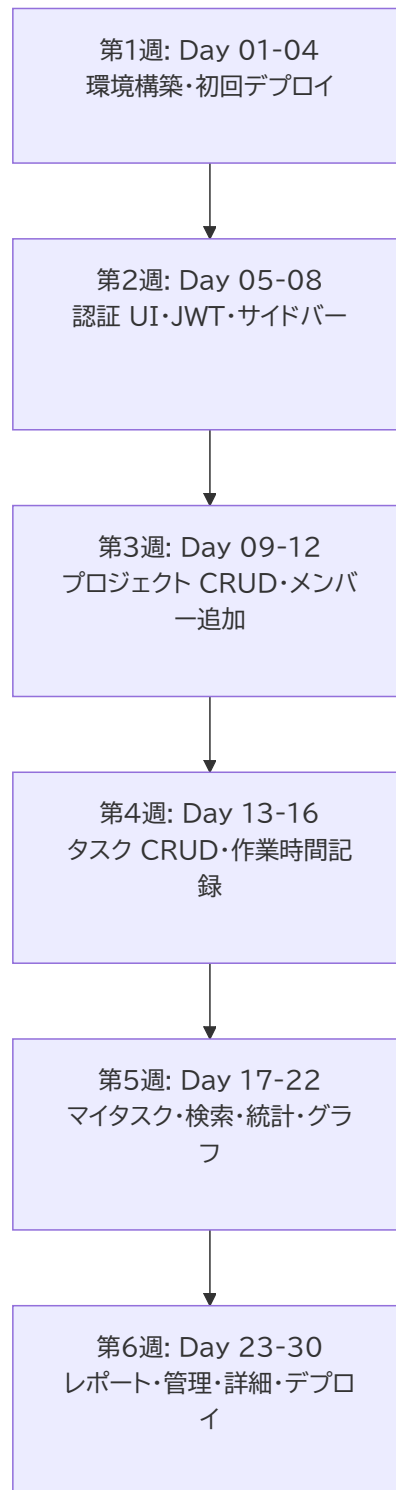


図 1: 30日間の歩み

この図は思い出の一覧ではありません。今日のデプロイが成立する理由は各週の成果物がそのまま本番の部品になるからです。第1週で GitHub と Vercel をつないだので今日は `git push` するだけでビルドが始まります。第2週で作った JWT の署名鍵は本番でも同じ役割を持つ環境変数として登録します。第3週から第5週で書いた画面と API は接続先の DB を差し替えるだけで本番でも同じコードのまま動きます。既存の画面や API のコードには手を入れません。今日は公開先を設定し、`next.config.ts` にセキュリティヘッダーを追加します。Step 4 では本番 DB の接続先を確認してスキーマを反映する補助スクリプトも作ります。

やること / やらないこと

やること	やらないこと
環境変数を Vercel に設定	独自サーバー構築
ローカル開発用 DB を Docker で確認	AWS/GCP のセットアップ
Vercel にデプロイ	ドメイン購入
本番動作確認	負荷テスト

ローカル DB と本番 DB の違い: Docker の PostgreSQL はローカル開発専用です。本番では Neon(ネオン。PostgreSQL をクラウド上で提供するサービス)などのマネージド DB を使います。Vercel Marketplace から Neon を連携すると、本番 DB の接続文字列が `DATABASE_URL` として自動で設定されます。Supabase などの別サービスは自動追加される変数名が異なるため、発行された接続文字列をこのアプリの `DATABASE_URL` に設定します。

新しく学ぶ概念

概念	読み方	役割	例え
Vercel	ヴァーセル	ホスティングサービス	レンタルキッチン
環境変数	かんきょうへんすう	設定情報の外部管理	店の裏の金庫
CI/CD	シーアイシーディー	自動ビルド・デプロイ	自動配送システム
Production	プロダクション	本番環境	実店舗の営業

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 1	本番用の環境変数を準備	5分
Step 2	本番前のローカル最終確認	5分
Step 2.5	セキュリティヘッダーを設定する	6分
Step 3	Git の公開準備	3分
Step 4	Vercel を設定してプッシュ	25分
Step 5	本番環境の動作確認	7分
Step 6	30日間の学習サマリー	7分

ステップ	作業内容	読む時間の目安
Step 7	技術スタックの振り返り	5 分
Step 8	次のステップとリソース	5 分

読む時間の合計(仮): 約 68 分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 1: 本番用の環境変数を準備(読む目安: 5 分)

ゴール: Vercel にデプロイするための環境変数を準備します。

必要な環境変数

変数名	値の例	用途
DATABASE_URL	<code>postgresql://user:pass@host:5432/db</code>	DB 接続(本番用)
JWT_SECRET	32 文字以上のランダムな秘密鍵	JWT の HMAC 署名鍵

`NODE_ENV` は Vercel が自動で `production` に設定するため手動設定は不要です。

Day 30: 完成版を公開

本番用の `DATABASE_URL` はクラウド DB サービスで用意します。Vercel Marketplace で Neon を選び、データベースを対象プロジェクトに接続すると `DATABASE_URL` が自動で追加されます。Supabase などの別サービスで作る場合は、発行された接続文字列をこのアプリ用の `DATABASE_URL` に設定します。Day 04 の初回デプロイ時に設定済みなら、その接続文字列をそのまま使います。

環境変数はまず公開先の Production に登録します。ブランチの Preview デプロイも使う場合は、同じ変数名を Preview にも登録します。 `JWT_SECRET` は環境ごとに別の値を生成してください。 `DATABASE_URL` を共有すると、Preview で作成・変更・削除したデータも本番に反映されます。本番データに影響させたくない場合は、Preview 専用の DB 接続先を用意します。

シークレットキーの生成:

```
# filepath: ターミナル
# ランダムなシークレットキーを生成
openssl rand -base64 32
# 出力例: K7x3mP9q...(これをコピー)
```

確認ポイント:

- 44 文字程度のランダム文字列が表示されました
- コピーして安全な場所にメモしました

`JWT_SECRET` は JWT トークンの HMAC 署名に使う秘密鍵です。

`openssl rand -base64 32` は32バイト(Base64で44文字)の鍵を生成します。

次に`.env.example` の主要変数(ローカル参考)を確認します。

```
# filepath: .env.example(主要部分の抜粋)

# ホスト側のポート設定

_DOCKER_COMPOSE_HOST_PORT_DB=25532

# DB接続文字列(ローカル開発用)

DATABASE_URL="postgresql://user:password@localhost:25532/taskapp?schema=public"

# JWT署名用の秘密鍵(32文字以上必須。本番では必ず変更)

JWT_SECRET="your-jwt-secret-key-32-chars-minimum-please-change"

# 本番URL(完成版の robots.txt 生成で使います。

# このカリキュラムでは robots.txt を作らないため、空のままで構いません)

# NEXT_PUBLIC_BASE_URL="https://your-app.vercel.app"
```

確認ポイント:

- `.env.example` の主要変数を確認できました
- `DATABASE_URL` の構造を理解しました

`.env.example` にはローカル開発用の設定が書かれています。`25532` は教材用 DB のホスト側ポートです。すでに使われている場合は

`_DOCKER_COMPOSE_HOST_PORT_DB` と `DATABASE_URL` のポート番号を同じ値に変更します。

本番では `.env` ファイルは使いません。Vercel のダッシュボードで環境変数を直接設定します。コードに秘密値を含めないのがセキュリティの基本です。

ローカルで `npm run build` を実行する前の準備: このプロジェクトは `prisma.config.ts` と `package.json` の `build` / `vercel-build` / `postinstall` で Prisma Client 生成を行うためローカルでも `DATABASE_URL` と `JWT_SECRET` が未設定だと build 時に失敗します。

この2つは Day 01 のセットアップで `.env` に用意済みです。中身が残っているかだけ確かめてから `npm run build` を実行してください。

VS Code のファイル一覧から `.env` を開き、`DATABASE_URL` と `JWT_SECRET` の行が残っているか確認してください。確認後はファイルを閉じ、ターミナルへ戻ります。

`.env` は `.gitignore` の `.env*` に当てはまるのでここへ書いた値は `git add` しても追跡対象になりません。中身の検証を担当するのは `src/lib/env.ts` の zod スキーマで、`DATABASE_URL` には URL の形を、`JWT_SECRET` には 32 文字以上を求めます。どちらかを満たさないと例外が投げられ、`npm run build` はページを 1 枚も出力せずに止まります。この検証は Vercel のビルドでも同じものが走ります。Vercel 側の環境変数に

`DATABASE_URL` と `JWT_SECRET` を入れ忘れると同じエラーでビルドが止まります。手元で一度通しておけば公開直前の本番ビルドログで初めてこのエラーを読む展開にはなりません。

確認ポイント:

- 2つの環境変数の値を準備できました

Step 2: 本番前のローカル最終確認(読む目安: 5分)

ゴール: 本番デプロイ前に、ローカルでアプリが正常に動くことを最終確認します。
`docker-compose.yml` の構成も把握しましょう。

次に `docker-compose.yml` の `db` サービス部分を抜粋して確認します。

実際のファイルにはテスト用 DB も定義されていますがここではメイン DB サービスだけを確認します。

```
# filepath: docker-compose.yml
services:
  db:
    image: postgres:16-alpine # 軽量版PostgreSQL
    environment:
      POSTGRES_USER: user      # DBユーザー名
      POSTGRES_PASSWORD: password # DBパスワード
      POSTGRES_DB: taskapp     # データベース名
    ports:
      - "${_DOCKER_COMPOSE_HOST_PORT_DB:-25532}:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 5s
      timeout: 5s
      retries: 5
```

`ports` の `25532:5432` はパソコン側の 25532 番をコンテナの中の PostgreSQL の 5432 番につなぐ指定です。`.env` の `DATABASE_URL` に書いたポート番号がこれとずれていると DB は動いているのに接続だけが拒否されます。`healthcheck` は `pg_isready` を 5 秒ごとに実行し、問い合わせを受け付けられる状態になって初めて `healthy` と表示します。起動直後の `npm run db:push` が接続エラーで止まったら、まず `healthy` と表示されるまで待ちます。`healthy` でも接続できない場合は、接続先のポート・ユーザー名・パスワードを確認してください。`volumes` はデータの保存先をコンテナの外へ逃がす指定で、これが無いとコンテナを作り直すたびに登録済みのユーザーが消えます。

確認ポイント:

- YAML のインデントがスペース 2 個で統一されています
- `ports` や `volumes` の値が 1 行で書かれています

docker-compose の主要設定

設定	値	意味
image	postgres:16-alpine	軽量版 PostgreSQL 16
POSTGRES_USER	user	DB ユーザー名
POSTGRES_PASSWORD	password	DB パスワード
POSTGRES_DB	taskapp	データベース名
ports	25532:5432	ホストからの接続ポート

DB の起動:

```
# filepath: ターミナル
# データベースを起動
docker compose up -d db

# 起動確認
docker compose ps

# マイグレーション実行
npm run db:push
```

確認ポイント:

- `docker compose ps` の db の STATUS に `(healthy)` が表示されました
- `npm run db:push` が成功しました

確認メモ: `docker compose ps` の `db` 行で STATUS の `(healthy)` と PORTS の `25532->5432/tcp` が見えればOKです。

`npm run db:push` はローカル確認用です。本番では `prisma migrate deploy` を使うのが一般的です。ただしこの30日教材では migration 履歴を作る手順を扱っていないため Step 4 で新しく作った教材用の本番 DB に限り `prisma db push` を1回実行します。既存データがある実務の DB ではこの手順をそのまま使わないでください。

Step 2.5: セキュリティヘッダーを設定する(読む目安: 6分)

ゴール: 公開したアプリがブラウザに守り方を伝えられるようにします。

ここまでのアプリにはセキュリティヘッダーが1つも入っていません。ヘッダーとは、ページ本体とは別にサーバーがブラウザへ渡す短い指示書のことです。これが無いとたとえば他人のサイトが自分のアプリを見えない枠として埋め込み、その上に偽のボタンを重ねる、といった手口を止められません。公開する前にここを埋めます。

`next.config.ts` を開き、`const nextConfig` の中へ `headers` を追加します。

```
// filepath: next.config.ts
// nextConfig の中に追加
async headers() {
  return [
    {
      source: '/*',
      headers: [
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options', value:
'nosniff' },
        { key: 'Referrer-Policy', value:
'origin-when-cross-origin' },
      ],
    },
  ];
},
```

`source: '/*'` は「すべてのURL」という意味で、アプリ全体に同じ指示を配ります。`X-Frame-Options: DENY` は他人のページの中へ自分のアプリを枠として埋め込むことを禁じます。`X-Content-Type-Options: nosniff` はブラウザがファイルの中身を見て種類を勝手に決め直すのをやめさせます。

`Referrer-Policy` は外部サイトへ移るときにどのページから来たかを細かく渡しすぎないようにします。

続けて通信と権限に関する3つを足します。

```
// filepath: next.config.ts(headers の配列に追加)

    {
      key: 'Strict-Transport-Security',
      value:
        'max-age=63072000; includeSubDomains; preload',
    },
    {
      key: 'Permissions-Policy',
      value:
        'camera=(), microphone=(), geolocation=(), browsing-topics=()',
    },
    { key: 'X-DNS-Prefetch-Control', value: 'off' },
```

Strict-Transport-Security は「次からは必ず暗号化した通信で来てください」という指示です。**max-age** は覚えておく秒数で、63072000 は2年ぶんにあたります。**includeSubDomains** はそのホストより下のサブドメインにも同じ指示を適用します。別の ***.vercel.app** にまで広がる指定ではありません。**preload** は HSTS プリロード一覧へ登録するときの必要条件です。この文字をヘッダーに書くだけで一覧へ登録されるわけではありません。今回の **vercel.app** の URL では、まず **max-age** によるそのホストの HTTPS 固定が効きます。

Permissions-Policy は、対応するブラウザで指定した機能の使用を制限するヘッダーです。このアプリで使わないカメラ・マイク・位置情報と、閲覧履歴から興味を推定する **browsing-topics** を無効にします。機能ごとにブラウザの対応状況が異なるため、MDNの対応表^{*1}も確認してください。

`X-DNS-Prefetch-Control` はリンク先の住所をあらかじめ引いておく動きを止めます。ここでいう住所引きは DNS(ドメイン名から通信先の番号を調べる仕組み) のことです。

確認ポイント:

- `next.config.ts` に `async headers()` を追加しました
- `npm run build` がエラーなく終わります
- 公開後にブラウザの開発者ツールの Network タブでページを選ぶと応答ヘッダーに `X-Frame-Options` が見えます

完成版のリポジトリにはこれに加えて `Content-Security-Policy` も入っています。読み込んでよい場所を種類ごとに列挙する指示で、効き目は大きいぶん、書き方を誤ると自分のアプリの画像やスクリプトまで止まります。まずは上の6つを入れて動く状態を保ったまま公開してください。

Step 3: Git の公開準備(読む目安: 3分)

ゴール: 30 日間の変更を漏れなくコミットし、公開直前の状態にします。

```
# filepath: ターミナル
# まず変更ファイルを確認
git status
```

1. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy#browser_compatibility

Day 30: 完成版を公開

公開の前に `git status` を読むのは載せてはいけないファイルをここで最後に一度だけ止められるからです。とくに `.env.local` が混ざったまま GitHub へ上がると `JWT_SECRET` を誰でも読める状態になります。この鍵さえ手に入れば他人でも有効なセッション Cookie を自分で作れます。Day 08 で組んだログイン確認は正しい署名として素通りさせてしまいます。一度 push した秘密鍵はあとからファイルを消しても履歴の中に残り続けます。だからこの1手だけは飛ばさないでください。

確認ポイント:

- `.env` ファイルが含まれていません
- `.gitignore` で秘密情報が除外されています

```
# filepath: ターミナル
# Day 04~29 で変更したアプリ用の場所を明示する
git add src prisma package.json package-lock.json
git add .env.example docker-compose.yml
git add next.config.ts tsconfig.json biome.json

# コミット対象と未ステージ差分を必ず確認する
git diff --cached --name-only
git status --short
```

一覧に `.env` や `.env.local` が無く、Day 04~29 の `src` と `prisma` が含まれることを確認してからコミットします。

```
# filepath: ターミナル
git commit -m "feat: 30日間の完成版"
```

確認ポイント:

- `.env` と `.env.local` が含まれていません
 - Day 29 の `src/server/api/routers/user.ts` も含まれます
 - `git status --short` の未追跡・未ステージを確認しました
-

Step 4: Vercel を設定してプッシュする(読む目安: 25分)

ゴール: Vercel にアプリをデプロイして公開します。

デプロイの流れ

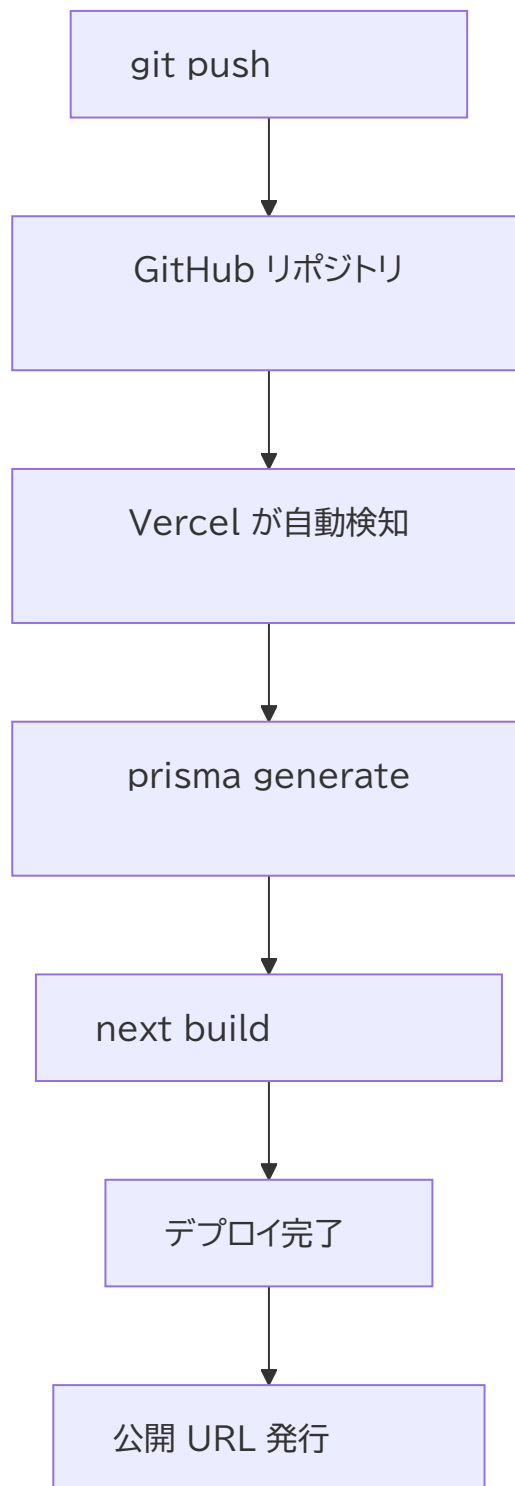


図2: デプロイの流れ

この流れのうち、自分の手で動かすのは左端の `git push` だけです。残りは Vercel が自動で進めます。ここで押さえておきたいのは環境変数を読む時点が `prisma generate` と `next build` だという点です。ビルドが始まったあとに Vercel の画面へ変数を足しても走っている最中のビルドはその値を知りません。だから順番は「環境変数を登録してから push」になります。逆にしてしまったときは Deployments タブから同じコミットを Redeploy すれば新しい値でビルドし直せます。

前提条件

Day 04 で Vercel 連携済みの場合は既存プロジェクトを開いてください。未連携の場合は以下の手順で準備します。

準備	手順
Vercel アカウント	vercel.com ^{*2} で GitHub 登録
プロジェクト Import	「Add New → Project」→ リポジトリ選択
Production Branch	Settings → Git で Day 04 から公開に使っているブランチ名を確認

Production の環境変数が使われるのは Production Branch のデプロイです。現在のブランチが異なると Preview デプロイになり、Production だけに登録した値は読み込まれません。Step 3 でコミットしたブランチ名と、Vercel の Production Branch が一致していることを確認してください。

Vercel で環境変数を設定:

2. <https://vercel.com>

Day 30: 完成版を公開

1. Vercel ダッシュボードにログイン
2. プロジェクトの Settings → Environment Variables
3. 以下を追加します

変数名	値	環境
DATABASE_URL	本番DBの接続文字列	Production
JWT_SECRET	Step 1 で生成した値	Production

確認ポイント:

- 2つの環境変数を Vercel に追加できました
- 各変数の「Environment」が Production になっています

Preview デプロイも使う場合は同じ2変数を Preview にも追加します。

Production だけを公開する場合はまず Production の設定と動作確認を完了させます。

2つの変数が何を左右するのかを、間違えたときの見え方で押さえておきます。

`DATABASE_URL` はどの PostgreSQL に読み書きするかを決めます。URL の形になっていなければ Step 1 で見た `src/lib/env.ts` の検証がビルドの途中で例外を投げます。Vercel のビルドはそこで失敗し、その回のデプロイは公開されません。公開 URL には前回のデプロイがそのまま残るので画面は 500 にならず、気付く場所はビルドログです。形は正しいのに空の DB を指していた場合は検証を通るのでデプロイまで進みます。画面も出ますが新規登録を押した瞬間にテーブルが無いというエラーが返ります。

`JWT_SECRET` はログイン Cookie の署名と検証に使う鍵です。この値は Production と Preview で別々にしてください。環境ごとに玄関が別にあるイメージです。練習用の玄関に合わせて作った鍵で、本物の玄関が開いてはいけません。両方に同じ値を入れると Preview のデプロイが発行した Cookie の署名を Production も正しいと判定します。ブラウザがその Cookie を自動で Production へ送るわけではありません。トークンの値が抜き取られ、本番へ送られた場合に検証を通ってしまうという意味です。Preview は動作確認用の環境で、気軽に作り直します。そこから本番の入口を通せる状態は避けます。

Preview を開いたときにログイン画面へ戻されるのは鍵が違うからではありません。Vercel は Production と Preview に別のホスト名を割り当てます。Day 07 で設定したログイン Cookie には `domain` を指定していないため Cookie は発行したホスト名にだけ送られます。Production で取った Cookie は Preview のホスト名には届かないので Preview では改めてログインすれば使えます。鍵をそろえてもこの動きは変わりません。

環境変数を先に設定できたら現在のブランチを GitHub へプッシュします。Day 03 と同じく、ブランチ名を固定しません。

```
# filepath: ターミナル
git push origin "$(git branch --show-current)"
```

この push で始まった Vercel デプロイが `Ready` になるまで待ってください。

確認ポイント:

- 現在のブランチの push が成功しました
- GitHub に Day 04～29 の変更が反映されました

Day 30: 完成版を公開

- 環境変数の設定後に始まったデプロイが **Ready** になりました

Vercel は GitHub と連携しているため **git push** するだけで自動的にビルドとデプロイが実行されます。これが CI/CD です。

ビルドスクリプトの確認:

package.json の **scripts** を確認しましょう。

スクリプト名	コマンド	用途
build	prisma generate && next build	通常ビルド
vercel-build	prisma generate && next build	Vercel 用ビルド

このプロジェクトでは **build** と **vercel-build** のどちらも **prisma generate && next build** を実行します。実際に選ばれたコマンドは Vercel のビルドログで確認してください。Prisma Client の生成後に Next.js のビルドが始まります。

確認ポイント:

- ビルドスクリプトの内容を理解できました
- Vercel のビルドログでエラーがありません
- デプロイ URL が発行されました

確認メモ: Vercel ダッシュボードの「Deployments」タブで最新デプロイが **Ready** になっていればOKです。

本番 DB に Prisma スキーマを反映する

デプロイが **Ready** でも、本番 DB が空のままだと登録やログインなど、DB を使う操作は失敗します。Vercel CLI で現在のフォルダを既存プロジェクトへ紐づけ、本番環境変数を一時的にコマンドへ渡してスキーマを反映します。

この手順は、Day 04 または Step 1 で用意した教材用の新規・空の DB に限ります。使う DB を管理画面で開き、表やデータをまだ入れていないことを確認してください。Neon では対象のブランチを選び、**Postgres database** → **Tables** でデータベースと **public** を選びます。**public** は、このアプリの表を置くスキーマ (DB 内で表をまとめる場所) です。既存の表がある場合は続行せず、接続先を確認します。この手順のために表を削除して空にする必要はありません。

取得に失敗したまま別の DB に接続しないよう、途中で失敗したら停止するスクリプトを使います。Vercel で Sensitive (秘密値の取得を制限する設定) を有効にすると、接続先の代わりに **[SENSITIVE]** が返ります。その場合は設定を有効にしたまま、DB の管理画面から接続 URL を取得して、このスクリプトの入力欄へ渡します。取得したファイルは dotenv (環境変数ファイルを読み取るライブラリ) で値として読み込みます。シェルのコマンドとして実行しないため、値に **\$()** などがあっても実行されません。

VS Code で **scripts/apply-production-schema.mjs** を新規作成し、次のコードを保存してください。**.mjs** は Node.js で **import** を使う JavaScript ファイルの拡張子です。一時ファイルの削除、秘密値の非表示入力、取消時の後片付けを1回の実行にまとめています。削除や取消の処理を貼り忘れないよう、以下は分割せず1ファイルとしてコピーしてください。コード内の **raw** は、入力文字を画面へ表示せずに受け取る端末の状態を指します。

Day 30: 完成版を公開

```
// filepath: scripts/apply-production-schema.mjs

import { spawnSync } from 'node:child_process';
import { randomBytes } from 'node:crypto';
import { mkdtempSync, readFileSync, rmSync } from
'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { StringDecoder } from 'node:string_decoder';
import { parse } from 'dotenv';

function readSecret(
  message =
    'DB 管理画面の接続 URL を貼り付けて Enter' +
    '(表示されません。Ctrl+C / Esc で取消): ',
) {
  const input = process.stdin;

  if (!input.isTTY || !process.stdout.isTTY || typeof
input.setRawMode !== 'function') {
    throw new Error(
      ('秘密値の入力には対話できる端末が必要です。' +
        'パイプや出力転送を外して実行してください。'),
    );
  }

  return new Promise((resolve, reject) => {
    const previousRaw = input.isRaw;
    const previousFlowing = input.readableFlowing;
    const decoder = new StringDecoder('utf8');

    let value = '';
```

```

let settled = false;

const finish = (error) => {

  if (settled) return;

  settled = true;

  input.removeListener('data', onData);

  input.removeListener('error', onFailure);

  input.removeListener('end', onFailure);

  input.removeListener('close', onFailure);

  process.removeListener('SIGINT', onCancel);

  try {

    input.setRawMode(previousRaw);

    if (previousFlowing) input.resume();

    else input.pause();

    process.stdout.write('\n');

  } catch {

    error = new

Error('端末の入力状態を戻せませんでした。端末を開き直してください。');

  }

  if (error) reject(error);

  else resolve(value);

  value = '';

};

const onCancel = () => finish(new

Error('DB への反映を中止しました。'));

const onFailure = () =>

  finish(new

Error('秘密値を読み取れませんでした。DB への反映を中止しました。'));

const onData = (chunk) => {

  const text = typeof chunk === 'string' ? chunk :

```

Day 30: 完成版を公開

```
decoder.write(chunk);

    if (['\u0003', '\u0004', '\u001b'].some((key) =>
text.includes(key))) return onCancel();

    const newline = text.search(/[\r\n]/u);

    const body = newline < 0 ? text : text.slice(0,
newline);

    if (newline >= 0 && !/^(?:\r\n|\r|\n)$/u.test(
text.slice(newline))) {

        return finish(new
Error('入力は1行だけにしてください。DB への反映を中止しました。'));

    }

    for (const character of body) {

        if (character === '\b' || character === '\u007f')
value = [...value].slice(0, -1).join('');

        else if (character.codePointAt(0) < 32)
return onFailure();

        else value += character;

    }

    if (value.length > 16384) return onFailure();

    if (newline >= 0) finish();

};

input.on('data', onData);
input.on('error', onFailure);
input.on('end', onFailure);
input.on('close', onFailure);
process.on('SIGINT', onCancel);

try {

    input.setRawMode(true);

    input.resume();
```

```

        process.stdout.write(message);
    } catch {
        onFailure();
    }
});
}

function run(args, env = process.env, stdio = 'inherit') {
    const result = spawnSync('npx', args, { stdio, env,
shell: false });
    if (result.error || result.status !== 0) {
        throw new
Error('コマンドが失敗しました。直前の表示を確認してください。');
    }
}

let directory;
try {
    if (process.platform === 'win32')
        throw new
Error('Windows では WSL2 の Ubuntu ターミナルから実行してください。');

    run(['vercel', 'link']);

    // 作成した専用フォルダだけを最後に削除するためです。
    directory = mkdtempSync(join(tmpdir(),
'task-app-production-'));

    process.stdout.write(`一時フォルダ: ${directory}\n`);

    const file = join(directory, 'production.env');

    run(['vercel', 'env', 'pull', file,
'--environment=production']);

```

Day 30: 完成版を公開

```
const values = parse(readFileSync(file));

// 確認待ちや DB 反映中に中断しても、接続情報ファイルを残さないためです。

rmSync(directory, { recursive: true, force: true });

if (!values.DATABASE_URL?.trim()) {

  throw new Error(('Production の DATABASE_URL が' +

    'ありません。' +

    'Vercel の設定を確認してください。'));

}

if (values.DATABASE_URL.trim() === '[SENSITIVE]') {

  process.stdout.write(

    ('Vercel の Sensitive 設定は変更しません。' +

      'DB 管理画面で教材用 DB の' +

      '接続 URL を確認してください。\\n'),

  );

  values.DATABASE_URL = await readSecret();

}

let target;

try {

  if (

    [...values.DATABASE_URL].some(

      (character) => character.codePointAt(0) < 32 ||

character === '\\u007f',

    )

  )

    throw new Error();

  target = new URL(values.DATABASE_URL);

} catch {

  throw new
```

```

Error('DATABASE_URL の形式を確認してください。値は共有しないでください。');

}

if (
  !['postgres:', 'postgresql:'].includes(
target.protocol) ||
  !target.hostname ||
  target.pathname.length < 2
) {
  throw new
Error('PostgreSQL のホスト名と DB 名が必要です。');
}

process.stdout.write(`接続先ホスト: ${
target.host} DB: ${target.pathname.slice(1)}\n`);

// 接続 URL と一緒に貼り付けられた別の行を、反映の確認に使わないためです。

const confirmation = `apply ${randomBytes(
4).toString('hex')}`;

const answer = await readSecret(

`上のホスト・DB が教材用の新規・空の DB と確認できたら ${
confirmation} と入力して Enter(表示されません。Ctrl+C / Esc で取消): `,

);

if (answer !== confirmation) throw new
Error('確認が一致しないため、DB への反映を中止しました。');

// 親の接続先やローカル .env より、今回取得した接続先を優先するためです。

const env = { ...process.env, DATABASE_URL:
values.DATABASE_URL };

delete env.DOTENV_CONFIG_OVERRIDE;

```

Day 30: 完成版を公開

```
// 未導入の Prisma の自動取得と、
// 遅い入力によるデータ損失への同意を止めるためです。
run(['--yes=false', 'prisma', 'db', 'push',
'--skip-generate'], env, [
  'ignore',
  'inherit',
  'inherit',
]);
} catch (error) {
  console.error(error.message);
  process.exitCode = 1;
} finally {
  if (directory) rmSync(directory, { recursive: true,
force: true });
}
```

スクリプトを実行して接続先を確認する

VS Code の「ターミナル」→「新しいターミナル」を開き、プロジェクトのルートで次を実行します。Windows では Day 01 と同じく、WSL2 の Ubuntu ターミナルを使ってください。このスクリプトは TTY(文字を入力して応答できる端末)で使います。コマンドへ `|` や `>` を付けず、そのまま実行してください。

```
# filepath: ターミナル
node scripts/apply-production-schema.mjs
```

最初の案内では、Day 04 から使っている既存の Vercel プロジェクトを選びます。初回に `Ok to proceed? (y)` と出たら、導入対象が `vercel` であることを確認して `y` を入力し、完了を待ちます。これは CLI の導入確認です。DB へ反映する確認は、この後に別の案内が出ます。

Sensitive の入力案内が出た場合:

1. ブラウザで Neon にログインし、Production の `DATABASE_URL` を用意したプロジェクトを開きます。
2. `Connect` を押し、`Connect to your branch` で本番と同じブランチ、Compute、データベース、ロールを選びます。
3. `Connection pooling` をオフにし、直接接続用の URL をコピーします。
4. ターミナルの「接続 URL を貼り付けて Enter」という案内へ戻り、コピーした URL 1 行だけを貼り付けて Enter を押します。

Prisma のスキーマ反映には直接接続を使います。ブランチは DB の状態を分けて管理する単位、Compute は接続を受け付ける実行環境です。ロールは DB 接続に使う利用者を指します。Day 04 または Step 1 で接続 URL を取得した DB の情報と照合してください。Neon の画面が変わった場合は、公式の接続案内^{*3}で `Connect` の場所を確認します。

コピーするのはパスワードが入った接続 URL です。`psql`、`DATABASE_URL=`、URL を囲む引用符は含めません。パスワードを星印で隠した見本は使わず、`sslmode` など `?` 以降の指定も残してください。`%`、括弧、記号を自分で置き換える必要はありません。値はこのスクリプトが入力を待っている間だけ貼り付けます。通常のコマンド入力欄や質問文には貼らないでください。

3. <https://neon.com/docs/connect/connect-from-any-app>

Day 30: 完成版を公開

入力した URL の文字は表示されません。星印も出ません。何も出なくても入力を受け付けています。貼り直したい場合は Ctrl+C または Esc で中止し、上の `node` コマンドからやり直します。強制終了後に通常のコマンド入力も見えなくなった場合は、その端末を閉じ、新しいターミナルを開いてください。URL を取得できない場合や、本番と同じ DB か分からない場合も中止してください。DB を管理する人には、プロジェクト名と確認したい内容だけを伝えます。接続 URL は添付しません。

Sensitive の案内が出なければ、取得した URL から次の接続先表示へ進みます。どちらの経路でも、表示されたホスト名と DB 名を管理画面の接続情報と照合してください。手入力で直接接続を選んだ場合は、Neon の `Connect` に表示されている直接接続用の情報と照合します。このスクリプトは、DB が空かどうかや、手入力した値と Vercel の秘密値が同じ接続先かを自動判定しません。

接続先が教材用の新規・空の DB と確認できたら、端末に表示された `apply` と 8 桁の確認コードを入力します。間には半角スペースを1つ入れ、最後に Enter を押してください。確認コードは `0 ~ 9` と `a ~ f` からなる16進数です。たとえば `apply 7c2e9a41` と表示された回は、その文字列を入力します。この例をコピーせず、自分の端末に出たコードを使ってください。確認コードは実行するたびに変わります。URL と一緒に貼り付けた別の行が、反映への同意として扱われるのを防ぐためです。この確認入力も表示されません。文字列が一致すれば反映が始まり、一致しなければ中止します。判断できないときは Ctrl+C、Esc、または何も入力せず Enter で中止してください。

確認ポイント:

- Day 04 から使っている Vercel プロジェクトを選びました
- 表示された接続先が教材用の新規・空の DB と一致しました
- 自分の端末に表示された、その回の確認コードで反映を開始しました

- `prisma db push` に `Your database is now in sync` と表示されました

`mkdtempSync` は専用の一時フォルダを作成します。取得したファイルは読み込んだ直後に削除します。確認待ちや DB 反映中に中断しても、このファイルは残りません。

`finally` は取得や読み込みの途中でエラーが出たときにも、作成したフォルダを削除するための後片付けです。既存の `.env.production.local` は変更しません。今回選んだ `DATABASE_URL` は Prisma の子プロセス(このスクリプトから起動するコマンド)だけに渡します。ターミナルに以前設定した接続先があっても、今回選んだ値を優先します。`--skip-generate` は Prisma Client の再生成を省く指定です。DB へのスキーマ反映には Prisma Client が不要なので、ここでは生成しません。

`--yes=false` は、プロジェクトに Prisma が無いときに別のバージョンを自動導入させない指定です。Prisma の実行中は端末から入力を渡しません。確認後に届いた文字で、Prisma の別の警告へ同意しないためです。データ損失の警告が出ると反映を止めます。`--accept-data-loss` を追加して続行せず、DB の接続先を確認してください。成功メッセージを確認してから Step 5 に進みます。

止まったときの確認先:

表示・状況	次に行うこと
WSL2 の Ubuntu から実行するよう表示された	Windows の Ubuntu を開き、同じプロジェクトのルートで実行します

表示・状況	次に行うこと
対話できる端末が必要と表示された	VS Code のターミナルを開き、パイプや出力転送を付けずに実行します
<code>node</code> や <code>npx</code> が見つからない	Day 01 の Node.js と npm の導入手順を確認し、端末を開き直します
<code>npx canceled due to missing packages and no YES option</code> と表示された	必要なパッケージが見つからず、自動導入を止めた状態です。プロジェクトのルートで <code>npm ci</code> を実行し、成功してからスクリプトをやり直します

表示・状況	次に行うこと
<code>ENOTCACHED</code> と表示された	オフラインで使うキャッシュが不足しています。通信できることと npm の実行環境を確認し、プロジェクトのルートで <code>npm ci</code> を実行します。成功後にスクリプトをやり直します
Vercel の認証・プロジェクト選択で失敗した	ログインしているアカウントと、選んだ既存プロジェクトを確認します
接続 URL の形式で止まった	URL だけをコピーしたか、引用符や星印の見本が混ざっていないかを確認します

表示・状況	次に行うこと
確認が一致しないと表示された	コマンドからやり直し、新しく表示された確認コードを入力します
DB 接続エラーになった	Production の接続先、選んだ DB、DB サービスの稼働状態を確認します
データ損失の警告で止まった	既存の表やデータがある DB を選んでいるか確認します

秘密値を含むファイルや接続文字列を質問文に貼らないでください。

`vercel env pull` で取得している途中の Ctrl+C、強制終了、停電では、接続情報ファイルは削除されずに残る場合があります。その場合は端末に表示された「一時フォルダ」の場所を確認し、そのフォルダを接続情報ファイルごと削除してください。

本番 DB 反映用スクリプトを履歴へ残す

接続先の確認とスキーマ反映が終わったら、作成したスクリプトだけをコミットします。スクリプトには接続 URL を書き込んでいません。取得した一時ファイルも削除済みです。それでも、先に `git status` で `.env` や一時ファイルが混ざっていないことをもう一度確かめます。

```
# filepath: ターミナル
# 本番 DB 反映用スクリプトだけをステージする
git status --short
git add -- scripts/apply-production-schema.mjs
git diff --cached --name-only
```

ここで止まり、`git diff --cached --name-only` の出力を確認します。`scripts/apply-production-schema.mjs` の1行だけなら次へ進みます。ほかのファイルが表示された場合は、そこで停止します。コミットや push はまだ行わず、Step 3 の秘密情報の確認へ戻ってください。とくに `.env`、`.env.local`、`production.env` はコミットしません。

確認できた場合だけ、次の2つを実行します。

```
# filepath: ターミナル
# 確認済みのスクリプトをコミットして現在のブランチへ push する
git commit -m "chore: add production schema helper"
git push origin "$(git branch --show-current)"
```

push が成功すると、GitHub から改めて取得した場合も同じスクリプトで接続先を確認できます。

確認ポイント:

- ステージしたファイルは `scripts/apply-production-schema.mjs` の1つだけです
- `.env`、`.env.local`、`production.env` はコミットに含まれていません
- 現在のブランチへの push が成功しました

実務では migration ファイルを Git で管理し、CI/CD から `prisma migrate deploy` を実行します。この教材を拡張してスキーマを変更するときは Prisma Migrate の導入を次の学習課題にしてください。

Step 5: 本番環境の動作確認(読む目安: 7分)

ゴール: 公開された URL で下のチェック表にある8項目の動作を確認します。管理者向け機能や権限別の試験は、この確認には含みません。

以下の `your-app-name` は、Step 4 で Vercel が発行した自分のホスト名に置き換えてください。

```
# filepath: ターミナル
# デプロイURLをブラウザで開く(macOS)
open https://your-app-name.vercel.app

# Ubuntu の場合
xdg-open https://your-app-name.vercel.app

# Windows (WSL2) の場合
explorer.exe https://your-app-name.vercel.app
```

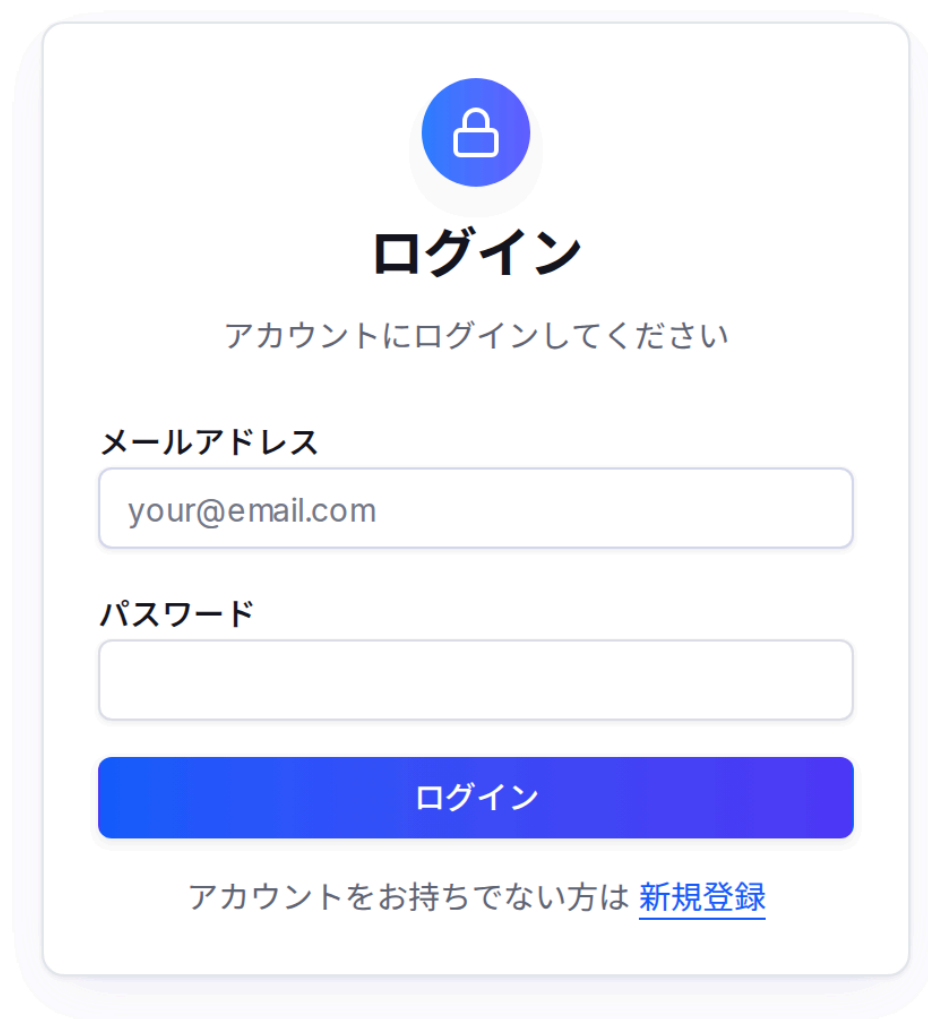
うまく開かないときはこの URL をコピーしてブラウザのアドレス欄に自分で貼っても構いません。ここでやりたいのは「その URL でアプリが見えること」の確認だけで、開き方は何でもよいからです。

ブラウザのアドレス欄が発行された `https://...vercel.app` になっていることを確認します。`localhost` は手元の開発サーバー、`vercel.app` はデプロイ先を指します。さらに別の回線でも開けるか確かめるなら Wi-Fi を切ったスマートフォンで同じ URL を開いてください。

確認ポイント:

- ブラウザでデプロイ URL が開けました
- ログインページが表示されます

【スクリーンショット】デプロイ先で開いたログイン画面は手元で見ていたものと同じです。下の画像は Day 05 で作ったログイン画面を手元で撮ったものですがデプロイ先でも同じ見た目になります。公開しても画面が変わらないのは同じコードが動いているからです。



The image shows a login screen with a white background and rounded corners. At the top center is a blue circular icon containing a white padlock. Below the icon is the title 'ログイン' (Login) in large black font. Underneath the title is the instruction 'アカウントにログインしてください' (Please login to your account) in smaller black font. There are two input fields: the first is labeled 'メールアドレス' (Email address) and contains the text 'your@email.com'; the second is labeled 'パスワード' (Password) and is empty. Below the input fields is a blue button with the text 'ログイン' (Login) in white. At the bottom, there is a link '新規登録' (New registration) in blue text, preceded by the text 'アカウントをお持ちでない方は' (If you do not have an account).

図3: ログイン画面。鍵アイコンの下に見出しと、メールアドレス・パスワードの入力欄、ログインボタン、登録リンクが並んでいる

本番環境チェックリスト

機能	確認内容	結果
ユーザー登録	<code>/register</code> で登録できる	☐
ログイン	<code>/login</code> で認証が通る	☐

機能	確認内容	結果
ダッシュボード	<code>/dashboard</code> が表示される	☐
プロジェクト	<code>/project</code> で作成・一覧表示	☐
タスク	<code>/task</code> で作成・ステータス変更	☐
レポート	<code>/report</code> で統計確認	☐
検索	<code>/search</code> でキーワード検索	☐
プロフィール	<code>/profile</code> で情報更新	☐

確認手順:

1. デプロイ URL にアクセス
2. `/register` で新規ユーザー作成
3. `/login` でログイン
4. `/dashboard` でダッシュボード確認
5. `/project` でプロジェクト作成
6. `/task` でタスク作成
7. `/report` で統計確認
8. `/search` で作ったタスクをキーワード検索
9. `/profile` で表示名を更新
10. ログアウト → 再ログイン

Day 30: 完成版を公開

ブラウザの DevTools を開き、Console にエラーが出ていないことも確認しましょう。Network タブで API レスポンスが 200 であることもチェックします。

【スクリーンショット】完成版のダッシュボード画面の表示を確認してください。本番では初期データを流さず、`/register` で作ったアカウントを使います。このアカウントの役割は一般ユーザーです。サイドバーは「ユーザー管理」の無い6項目になります。



図4: 完成したダッシュボード。左のサイドバーに6項目のメニューが並び、右に見出しとカードが出ている

Step 6: 30日間の学習サマリー(読む目安: 7分)

ゴール: 30日間で身につけたスキルを振り返ります。

```
# filepath: ターミナル
# これまでのコミット数を確認
git log --oneline | wc -l
# 作成したページ数を確認
find src/app -name "page.tsx" | wc -l
```

`git log --oneline` は1コミットを1行で出すので `wc -l` に渡すとそのまま件数になります。同じ日に複数回コミットすることもあるので、件数から作業日数は分かりません。Day 03 からの変更履歴を振り返る目安にします。下の `find` が数えているのは `page.tsx` というファイル名だけです。App Router では `page.tsx` を置いたフォルダの位置がそのまま URL になるのでこの件数は公開したアプリの画面数とほぼ一致します。数字が思ったより少なければフォルダを作っただけで `page.tsx` を置いていない場所が残っています。

確認ポイント:

- コミット総数を確認できました。毎日の継続を証明する数字ではないと理解しました
- 教材の手順で作る16個の `page.tsx` を確認できました。独自に画面を追加した場合は件数が増えます

週ごとの学習内容

週	Day	学んだこと
第1週	1-4	環境構築・初回デプロイ
第2週	5-8	認証 UI・JWT・サイドバー

週	Day	学んだこと
第3週	9-12	プロジェクト CRUD・メンバー追加
第4週	13-16	タスク CRUD・ステータス・作業時間記録
第5週	17-22	マイタスク・検索・統計・グラフ
第6週	23-30	レポート・管理・詳細・デプロイ

30日間で、教材スターターから機能を段階的に足して 16 ページのアプリを構築しました。フロントエンドからバックエンド、データベース設計からデプロイまで一貫して経験できました。

Step 7: 技術スタックの振り返り(読む目安: 5 分)

ゴール: このアプリで使った技術スタックを総復習します。

```
# filepath: ターミナル
# 主要パッケージのバージョンを確認
npm ls next react typescript prisma
```

`npm ls` が表示するのは `package.json` に書いた希望のバージョンではなく、`node_modules` へ実際に入った番号です。この番号を覚えておく価値は公式ドキュメントを読むときに出てきます。Next.js は 14 系と 15 系で書き方の変わった箇

所があります。手元が 15 系だと知らないまま古い記事のコードを写すとそのままでは動きません。`UNMET DEPENDENCY` と表示された場合は必要なパッケージが入っていない状態です。`npm install` をやり直してからもう一度本番のビルドを確認してください。

確認ポイント:

- 各パッケージのバージョンが表示されました
- 各技術の役割を説明できます

フロントエンド技術

セットアップでは教材で指定したバージョン範囲のパッケージを導入します。実際に入った番号は `npm ls` と `package-lock.json` で確認してください。

技術	バージョン	役割
Next.js	15.5.24	フレームワーク(App Router)
React	18.3.1	UI ライブラリ
TypeScript	5.x	型安全な JavaScript
shadcn/ui	—	UI コンポーネント
Tailwind CSS	v4	ユーティリティ CSS
Recharts	3.x	グラフ・チャート

バックエンド技術

技術	バージョン	役割
tRPC	11.x	End-to-End 型安全 API
Prisma	6.x	ORM(DB 操作)
PostgreSQL	16	データベース
jose	—	JWT トークン生成・検証
bcryptjs	—	パスワードハッシュ化

開発ツール

技術	バージョン	役割
Biome	2.x	リンター・フォーマッター
Vitest	3.x	テストフレームワーク
Docker	—	コンテナ(PostgreSQL)
Vercel	—	ホスティング・CI/CD

この技術スタックは2024-2026年のモダン Web 開発で広く使われています。ここで学んだ知識は実務でも活かれます。

Step 8: 次のステップとリソース(読む目安: 5分)

ゴール: 今後の学習の方向性と参考リソースを確認します。

```
# filepath: ターミナル
# プロジェクトのコード行数を確認
find src \( -name "*.ts" -o -name "*.tsx" \) \
  | xargs wc -l | tail -1
```

`find` で集めた `.ts` と `.tsx` を `xargs` が `wc -l` へまとめて渡し、最後の `tail -1` がファイルごとの内訳を捨てて合計行だけを残します。この数字は成績ではありません。作ったコードの量を確認するために使います。次に何を作るか迷ったら、`src` 配下のファイルを開き、自分の言葉で説明できるか確かめましょう。説明できない箇所が、次に読み返す場所です。

確認ポイント:

- `src` 配下のコードの総行数を確認しました
- 次の学習目標を決められました

次に挑戦できること

カテゴリ	内容	難易度
機能追加	通知システム	中
機能追加	ファイル添付	中
機能追加	カレンダービュー	中～高
性能改善	キャッシュ戦略	中
性能改善	コード分割の深掘り	中
品質向上	E2E テスト充実	中
インフラ	CI/CD パイプライン	中

カテゴリ	内容	難易度
------	----	-----

新技術	WebSocket リアルタイム通信	高
-----	--------------------	---

公式ドキュメント

技術	URL
----	-----

Next.js	https://nextjs.org/docs ^{*4}
tRPC	https://trpc.io/docs ^{*5}
Prisma	https://www.prisma.io/docs ^{*6}
shadcn/ui	https://ui.shadcn.com ^{*7}
Tailwind CSS	https://tailwindcss.com/docs ^{*8}
Vitest	https://vitest.dev ^{*9}
Biome	https://biomejs.dev ^{*10}

学習リソース

リソース	URL
------	-----

React 公式	https://react.dev ^{*11}
----------	--

-
4. <https://nextjs.org/docs>
 5. <https://trpc.io/docs>
 6. <https://www.prisma.io/docs>
 7. <https://ui.shadcn.com>
 8. <https://tailwindcss.com/docs>
 9. <https://vitest.dev>
 10. <https://biomejs.dev>
 11. <https://react.dev>

リソース	URL
TypeScript Handbook	https://www.typescriptlang.org/docs ^{*12}
MDN Web Docs	https://developer.mozilla.org ^{*13}

公式ドキュメントが最も正確で最新の情報源です。困ったときはまず公式ドキュメントを読みましょう。

Pro パターンで書こう(振り返り画面を例に、Server Component を標準にする)

この画面は説明のための例で、完成版には含まれません。

静的な表示部分を Server Component に残し、状態やクリック処理が必要な部品だけを Client Component に分けます。なぜ直前の1文の書き方をするのか、**Before/After** で見比べてみましょう。

12. <https://www.typescriptlang.org/docs>

13. <https://developer.mozilla.org>

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(実ファイルには対応しません)

'use client';

import { useState } from 'react';

const CURRICULUM_SUMMARY = [
  { label: '認証', value: 'JWT ログイン' },
  { label: 'プロジェクト', value: 'CRUD + メンバー管理' },
  { label: 'タスク', value: 'CRUD + 一括操作' },
  { label: '公開', value: 'Vercel デプロイ' },
];

export default function GraduationPage() {
  const [copied, setCopied] = useState(false);

  const handleShare = async () => {
    await
    navigator.clipboard.writeText('Task-App 30日間カリキュラムを完走しました');
    setCopied(true);
  };

  return (
    <main className="mx-auto max-w-4xl space-y-6 p-8">
      <h1 className="text-3xl font-bold">
Task-App 30日間ハンズオン修了</h1>
      <div className="grid gap-4 md:grid-cols-2">
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

ここまでで目を留めてほしいのは1行目の `'use client'` がファイル全体にかかっている点です。この宣言は行や関数ではなく、ファイル単位で効きます。だから下に続く振り返りカードも、動きを持たない見出しも、まとめてブラウザ側へ送られます。

```

    { /* filepath: 読み比べ用サンプル(続き・実ファイル
には対応しません) */ }

    {CURRICULUM_SUMMARY.map((item) => (
      <section key={item.label} className="rounded-lg
border p-4">
        <p className="text-sm text-muted-foreground">{
item.label}</p>
        <p className="text-lg font-semibold">{
item.value}</p>
      </section>
    ))}
  </div>
  <button type="button" className="rounded-md border
px-4 py-2" onClick={handleShare}>
    {copied ? 'コピー済み' : '卒業メッセージをコピー'}
  </button>
</main>
);
}

```

このコードの問題点:

Day 30: 完成版を公開

- ほとんど静的な振り返り画面まで Client Component になり、不要な JavaScript が増えます
- `useState` が必要なのはコピーボタンだけなのにページ全体がブラウザ実行前提になります
- 最終日の構成確認で「どこが対話部分か」が見えにくくなります

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)
import { ShareGraduationButton } from
'./share-graduation-button';
const CURRICULUM_SUMMARY = [
  { label: '認証', value: 'JWT ログイン' },
  { label: 'プロジェクト', value: 'CRUD + メンバー管理' },
  { label: 'タスク', value: 'CRUD + 一括操作' },
  { label: '公開', value: 'Vercel デプロイ' },
];

export default function GraduationPage() {
  return (
    <main className="mx-auto max-w-4xl space-y-6 p-8">
      <h1 className="text-3xl font-bold">
Task-App 30日間ハンズオン修了</h1>
      <div className="grid gap-4 md:grid-cols-2">
        {CURRICULUM_SUMMARY.map((item) => (
          <section key={item.label} className="rounded-lg
border p-4">
            <p className="text-sm text-muted-foreground">{
item.label}</p>
```

```
        <p className="text-lg font-semibold">{  
item.value}</p>  
    </section>  
    )}  
</div>  
  
<ShareGraduationButton  
text="Task-App 30日間カリキュラムを完走しました" />  
</main>  
  
);  
}
```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

Day 30: 完成版を公開

ここまでがサーバー側に残る部分です。'use client' が消え、useState の取り込みも無くなりました。カードを並べる .map() は Before とまったく同じままです。動かしたのはボタン1個だけで、置き換えた `<ShareGraduationButton />` の中身は次のブロックで別ファイルとして作ります。

```
// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)

'use client';

import { useState } from 'react';

type ShareGraduationButtonProps = {
  text: string;
};

export function ShareGraduationButton({ text }:
ShareGraduationButtonProps) {
  const [copied, setCopied] = useState(false);

  const handleShare = async () => {
    await navigator.clipboard.writeText(text);
    setCopied(true);
  };

  return (
```

```

<button type="button" className="rounded-md border
px-4 py-2" onClick={handleShare}>
  {copied ? 'コピー済み' : '卒業メッセージをコピー'}
</button>

```

読み比べ用: ここは写経しません。続けてコードを読み進めましょう。

'use client' はこの小さなボタンのファイルだけに付きました。コピー済みかどうかを覚える `useState` も、ここに閉じ込められています。親のページは Server Component のままなので、この例で追加したブラウザ側の処理はボタンの境界に収まります。React や Next.js の共通処理は別にあり、実際の配信量や表示速度はビルド結果を測らなければ分かりません。ここでは Client Component にする範囲だけを確認します。

```

// filepath: 読み比べ用サンプル(続き・実ファイルには対応しません)
);
}

```

このコードの強み:

- 静的な振り返り本文は Server Component のまま配信できます
- ブラウザで状態を持つのはコピーボタンだけになり、責務の境界が見えます
- 本番公開前の設計レビューで「client 化が必要な場所」を説明しやすい

覚えておきたいエッセンス

App Router では Server Component を標準にしてクリック・入力・ブラウザ API が必要な小さな部品だけを Client Component に切り出します。

完成コード全体

今日は `next.config.ts` の変更と、

`scripts/apply-production-schema.mjs` の新規作成をしました。環境変数の登録は Vercel の画面で行い、`.env.example` と `docker-compose.yml` は中身を確認しました。補助スクリプトは Step 4 に載せた完全なコードと見比べてください。ここには `next.config.ts` を再掲します。ヘッダーの入れ子が深く、どの階層へ貼るか迷いやすい場所なので、手元のコードと照合しましょう。

ファイル	役割	対応する Step
<code>next.config.ts</code>	ブラウザへ渡すセキュリティヘッダーの設定	Step 2.5
<code>scripts/apply-production-schema.mjs</code>	接続先を確認して本番 DB ヘスキーマを反映	Step 4 に全体を掲載

`next.config.ts`

このファイルは丸ごと置き換えません。もともと書かれている設定はそのまま残し、

`const nextConfig` の中へ以下の `headers` を足した状態が完成形です。

ヘッダー設定の開始部分:

```
// filepath: next.config.ts
// 完成版: ヘッダー設定の開始部分
async headers() {
  return [
    {
      source: '/*',
      headers: [
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options',
          value: 'nosniff' },
        { key: 'Referrer-Policy',
          value: 'origin-when-cross-origin' },
      ]
    }
  ]
}
```

入れ子が3段になっている理由はこの形が「どのURLに、どのヘッダーを配るか」の組を何通りでも書けるようにしてあるからです。いちばん外の配列が組の一覧、その中の `source` が対象のURL、内側の `headers` が配る中身です。今回は組が1つしかないので冗長に見えますがあとから管理画面だけ別の指示を配りたくなったときに2つ目の組を並べるだけで済みます。この `headers` 関数は、ヘッダー設定の配列をPromiseで返す形になっています。`async` を付けた関数は `return` した値をPromiseで返します。今回は `await` を書いていませんが、NextConfigが求める返り値の形に合わせて `async` を付けています。`await` を書くための指定ではない、という点を押さえておきましょう。

通信と権限のヘッダーと閉じカッコ:

```
// filepath: next.config.ts(同じファイルの続き)
// 完成版: 通信と権限のヘッダーと閉じカッコ

    {
      key: 'Strict-Transport-Security',
      value:
        'max-age=63072000; includeSubDomains; preload',
    },
    {
      key: 'Permissions-Policy',
      value:
        'camera=(), microphone=(), geolocation=(), browsing-topics=()',
    },
    { key: 'X-DNS-Prefetch-Control', value: 'off' },
  ],
},
];
},
```

前半の3つと違ってこちらは1つずつ改行して書いています。`value` が長いので1行に収めると読みにくく、`npm run fix` を実行しても Biome がこの形に整えます。閉じカッコが `]`、`}`、`]`、`}` の順に4段ぶん並ぶので貼り付けたあとは `const nextConfig` の閉じカッコが1つ余っていないかを確認してください。ここが合っていないと `npm run build` は設定を読む前に構文エラーで止まります。最後の `,` はこの `headers` のうしろに別の設定が並んでも壊れないようにするためのものです。

Content-Security-Policy はここに入っていない。Step 2.5 で触れたとおり、書き方を誤ると自分のアプリの画像やスクリプトまで止まるので公開を先に成立させるほうを取っています。上の6つで動く状態を確かめてから次の課題として足してください。

今日のまとめ

- ☐ 環境変数を Vercel に設定しました
- ☐ Docker で DB を起動できました
- ☐ セキュリティヘッダーを追加して応答で確認しました
- ☐ Git にプッシュしました
- ☐ Vercel にデプロイできました
- ☐ 本番環境で Step 5 の8項目を確認しました
- ☐ 30日間の学習を振り返りました
- ☐ 技術スタックを総復習しました
- ☐ 次のステップを決めました

つまずきポイント

エラー / 問題	原因	解決方法
ビルドが失敗する	型エラー・依存関係・環境変数など	Step 3 の「Deployments」で失敗したデプロイを開き、ビルドログの最初のエラーを確認。該当箇所を修正して再デプロイ

エラー / 問題	原因	解決方法
DB 接続エラー	DATABASE_URL が不正	接続文字列を再確認
JWT エラー	JWT_SECRET が未設定	openssl で生成して設定
ページが真っ白	JS エラー	DevTools Console を確認

今日学んだ用語

用語	意味
Vercel	Next.js に最適化されたホスティング
デプロイ	アプリを本番サーバーに配置する
CI/CD	自動ビルド・自動デプロイの仕組み
環境変数	アプリの設定を外部から注入する仕組み
Production	ユーザーが使う本番環境
マネージド DB	クラウド事業者が運用する DB

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. 公開したアプリを開くと手元の開発画面と同じ見た目でした。どこを見ればデプロイ先を開いていると確認できますか。

A. ブラウザのアドレス欄を確認します。発行された `https://...vercel.app` ならデプロイ先、`localhost` なら手元の開発サーバーです。別の回線からの確認には Wi-Fi を切ったスマートフォンを使えます。

Q2. `prisma db push` を本番の DB に対して実行してよいのはどんなときですか。

A. Day 04 または今日の Step 1 で作成した、この教材用の新規・空の本番 DB に対して1回だけです。実務では `prisma migrate deploy` を使います。この教材は migration の履歴を作る手順を扱っていないため中身が空の DB に限って `db push` で表を作ります。すでにデータが入っている DB に、この手順をそのまま持ち込むことはできません。

Q3. セキュリティヘッダーを6つ入れる一方で、`Content-Security-Policy` は入れていないのはなぜですか。

A. 効き目が大きいぶん、書き方を誤ると自分のアプリの画像やスクリプトまで止まるためです。まず6つを入れて動く状態を保ったまま公開し、そのあとの課題として足します。

追加課題: 公開先の別ページでもヘッダーを確かめる

設定ファイルに書いた指示が公開先まで届くことを確認します。理解チェック Q1 の URL 確認と、Day 26 の Network タブを組み合わせてみましょう。

前提は Step 5 の公開先でログインできることです。ブラウザのアドレス欄が自分に発行された公開 URL であることを確認します。

Day 30: 完成版を公開

ログイン中ならログアウトして `/login` を開きます。開発者ツールの Network タブを開き、そのページを再読み込みしてください。一覧からページ本体のリクエストを選び、Headers の Response Headers(サーバーから返ったヘッダー)を開いてください。

`x-content-type-options` が `nosniff` かを確認します。ログイン後に `/task` を開いて再読み込みし、ページ本体の応答でも同じ値を確認できれば成功です。表示上、ヘッダー名が小文字でも構いません。

値が無い場合は `next.config.ts` の `source` と、Vercel が公開したコミットを Step 4 で確かめてください。2 ページへ同じ設定が届く理由を、Step 2.5 の `source` から説明します。確認後は開発者ツールを閉じます。設定の変更や再デプロイはこの課題の操作には含めません。

卒業おめでとうございます

Task-App 30日間ハンズオンカリキュラム修了

卒業チェックリスト

以下の項目を確認して 30 日間の学びを振り返りましょう。

#	カテゴリ	できるようになったこと	学んだ Day
1	環境構築	<code>npm run dev</code> で アプリを起動できる	Day 01

#	カテゴリ	できるようになったこと	学んだ Day
2	UI 基礎	ダッシュボードにメッセージを追加できる	Day 02
3	Git	コミット・プッシュができる	Day 03
4	デプロイ基礎	ネットに公開できる	Day 04
5	認証 UI	ログイン・登録画面を作れる	Day 05-06
6	認証機能	JWT + Cookie の仕組みを説明できる	Day 07-08
7	API	tRPC でサーバー・クライアント通信ができる	Day 09-10
8	CRUD	プロジェクト・タスクの作成・編集・削除ができる	Day 10-16
9	機能拡張	マイタスク・コメント・検索を実装できる	Day 17-20

#	カテゴリ	できるようになったこと	学んだ Day
10	レポート	統計・グラフ・週次 レポートを表示できる	Day 21-23
11	管理機能	ユーザー一覧・プロフィール編集ができる	Day 24-25
12	品質管理	エラーページ・デバッグができる	Day 26
13	詳細・一括	プロジェクト詳細・タスク一括操作ができる	Day 27-28
14	仕上げ	ユーザー詳細・編集・本番デプロイができる	Day 29-30

作った機能を振り返る

本編では、画面を作り、入力を検証し、データを保存するコードを順に追加しました。公開まで確認できたら、ログインからタスク作成までを自分のアプリで試してみてください。

自分の言葉で説明できない箇所があれば、その Day に戻って読み直しましょう。

次のステップ

技術スタックの詳細は Step 7、次に挑戦できることの一覧は Step 8 を参照してください。学び続けること、作り続けることが大切です。

次のプロジェクトでも、ここで学んだスキルを活かしてさらに成長していきましょう。

Happy Coding

次に読むもの

- 前の日: Day 29^{*14}
- 全体の地図: 学びのロードマップ^{*15}
- 目次: カリキュラム目次^{*16}
- 詰まったとき: トラブルシューティング^{*17}
- 言葉の意味: 用語集^{*18}

奥付

タイトル

Day 30: 完成版を公開！

14. https://drive.google.com/file/d/1jxIONgRWWgaENJ1d3oE2OwQmG_uF7KPI/view?usp=drivesdk

15. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk

16. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>

17. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>

18. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。